

مستند فنی-آکادمیک ماژول‌های Auth، API Gateway و Users

1. مقدمه

در چارچوب این پروژه، یک پلتفرم دانشگاهی مبتنی بر معماری میکروسرویس پیاده‌سازی گردیده است. مطابق نیازمندی‌های تعریف‌شده، سامانه می‌بایست دارای درگاه ورودی واحد (API Gateway)، سرویس احراز هویت (Auth Service) مبتنی بر JWT و سرویس مدیریت کاربران (User Service) باشد. این مستند به تشریح طراحی و پیاده‌سازی این سه مؤلفه، همراه با مبانی نظری مرتبط، اختصاص دارد.

2. API Gateway

2.1. هدف و جایگاه معماری

API Gateway به‌عنوان نقطه‌ی ورود یکتای سامانه (Single Entry Point) طراحی شده است تا تمامی درخواست‌های کلاینت ابتدا وارد این لایه شده و سپس به میکروسرویس‌های پشت‌صحنه هدایت گردند. پیاده‌سازی گیتوی بر بستر FastAPI انجام شده و وظایف مشترک نظیر مسیریابی معکوس، اعمال سیاست‌های امنیتی، مدیریت CORS، ارتقای تاب‌آوری سامانه و بهبود کارایی در این لایه متمرکز گردیده است.

2.2. سازمان‌دهی پیاده‌سازی

در ساختار پوشه‌ی /API-GATEWAY، فایل main.py برای ایجاد برنامه و نصب میان‌افزارها و روترها در نظر گرفته شده است. پوشه‌ی /ROUTERS جهت تعریف روترهای پراکسی سرویس‌های مختلف ایجاد گردیده و پوشه‌ی UTILS شامل ابزارهای مشترک (Proxy، CircuitBreaker، AuthMiddleware و Services) است. این تفکیک با هدف افزایش خوانایی و نگهداشت‌پذیری سامانه صورت پذیرفته است.

2.3. پیکربندی CORS

به‌منظور پشتیبانی از کلاینت‌های ناهمگون، CORS در main.py فعال شده است. در این پیکربندی، allow_origins، allow_methods و allow_headers برابر "*" تنظیم گردیده‌اند. از منظر معماری، مدیریت CORS از جنس Cross-Cutting Concern بوده و استقرار آن در گیتوی سبب حذف تکرار منطق در سرویس‌های داخلی می‌شود.

2.4. میان‌افزار JWT در گیتوی

یک میان‌افزار اختصاصی جهت احراز هویت مبتنی بر JWT در گیتوی پیاده‌سازی شده است. در این میان‌افزار، مسیرهای عمومی (login/register/refresh) از مسیرهای محافظت‌شده تفکیک شده‌اند. برای مسیرهای محافظت‌شده، توکن از هدر Authorization استخراج و برای اعتبارسنجی به endpoint /auth/verify در سرویس Auth ارسال می‌گردد. در صورت عدم اعتبار توکن، پاسخ 401 و در صورت عدم دسترسی سرویس Auth، پاسخ 503 بازگردانده می‌شود. این رویکرد در دسته‌ی Centralized Token Verification قرار می‌گیرد.

2.5. مسیریابی و پراکسی

برای هر میکروسرویس، یک Router پراکسی تعریف شده است. مسیرهای /api/auth/ به سرویس Auth و مسیرهای /api/users/ به سرویس Users ارجاع داده می‌شوند. ارسال درخواست‌ها به سرویس مقصد از طریق تابع مشترک Proxy.forward انجام می‌شود تا منطق هدایت یکپارچه و قابل توسعه باقی بماند. در نسخه‌ی نهایی، می‌بایست از عدم تکرار prefix در سطح Router و include_router اطمینان حاصل گردد.

2.6. Cache و Circuit Breaker

به منظور افزایش تاب‌آوری، الگوی Circuit Breaker در لایه‌ی گیت‌وی به‌کار گرفته شده است. برای هر سرویس یک breaker مستقل با آستانه‌ی سه خطای متوالی و `reset_timeout` برابر ۱۰ ثانیه در نظر گرفته شده است. در حالت OPEN، درخواست‌ها به‌صورت سریع fail می‌شوند تا از بروز سقوط آبنشاری جلوگیری شود و پس از سپری‌شدن زمان تعیین‌شده، مدار وارد وضعیت HALF-OPEN می‌گردد. همچنین، برای بهبود کارایی، پاسخ‌های موفق GET در لایه‌ی گیت‌وی cache می‌شوند تا latency کاهش یافته و بار سرویس‌های داخلی کمتر گردد.

3. Auth Service

3.1. هدف سرویس

سرویس Auth به‌منظور مدیریت Authentication سامانه طراحی و پیاده‌سازی شده است. این سرویس عملیات ثبت‌نام، ورود، صدور Access Token و Refresh Token، نوسازی توکن‌ها و صحت‌سنجی JWT برای مصرف در API Gateway را بر عهده دارد.

3.2. ساختار پیاده‌سازی

در `/AUTH-SERVICE/APP`، فایل `main.py` برای ساخت برنامه و پیکربندی CORS و `prefix`ها قرار دارد. مدل‌های پایگاه داده در `models.py` تعریف شده‌اند، عملیات DB در `crud.py` پیاده‌سازی گردیده، منطق امنیتی (`JWT` و `bcrypt`) در `security.py` آمده و نقش‌ها در `schemas.py` مشخص شده‌اند. `endpoint`های سرویس نیز در `ROUTERS/Auth.py` مستقر هستند.

3.3. مدل داده و نقش‌ها

نقش‌های `student`، `teacher` و `admin` به‌صورت Enum تعریف شده‌اند. مدل `User` شامل شناسه، ایمیل یکتا، گذرواژه‌ی هش‌شده، نام کامل، نقش و وضعیت فعال‌بودن است. همچنین مدل `RefreshToken` برای نگهداری توکن‌های بلندمدت با فیلدهای `expires_at` و `user_id` ایجاد گردیده تا امکان ادامه نشست کاربر فراهم شود.

3.4. هش‌کردن گذرواژه

برای ذخیره امن گذرواژه‌ها، الگوریتم `bcrypt` مورد استفاده قرار گرفته است. در این روش، گذرواژه‌ها به‌صورت یک‌طرفه هش می‌شوند و به دلیل بهره‌گیری از `salt` داخلی، در برابر حملات `rainbow table` مقاوم هستند.

3.5. JWT و Claims

Access Token با استفاده از `SECRET_KEY` و `ALGORITHM` امضا می‌شود و Claims کلیدی از جمله `iat`، `user_id/sub`، `role`، `exp` و `payload` قرار می‌گیرند. این طراحی باعث `self-contained` بودن توکن و Stateless ماندن سرویس‌های سامانه می‌گردد.

3.6. Endpoint های اصلی

Register: در `endpoint` ثبت‌نام، تکراری نبودن ایمیل بررسی شده، گذرواژه هش می‌گردد و کاربر در DB ایجاد می‌شود. پس از آن، همگام‌سازی داخلی با سرویس Users از طریق `POST /api/users/internal/sync` و `POST /api/users/internal/sync` و `Service-Key` انجام می‌پذیرد.
Login: اعتبارسنجی اطلاعات ورودی انجام شده، Access Token و Refresh Token تولید می‌شود و Refresh Token در DB ذخیره می‌گردد.
Refresh: اعتبار Refresh Token در DB بررسی شده و در صورت معتبر بودن، Access Token جدید صادر می‌شود.
Verify: توکن دریافت‌شده `decode` شده و اعتبار آن برای مصرف گیت‌وی گزارش می‌گردد.

4. User Service (Users)

4.1. هدف سرویس

سرویس Users جهت مدیریت داده‌های پروفایل و عملیات کاربری (مجزا از credentialها) پیاده‌سازی گردیده است. این تفکیک امکان سبک‌سازی سرویس Auth و توسعه‌ی مستقل حوزه پروفایل کاربران را فراهم می‌سازد.

4.2. ساختار و مدل User

در /USER-SERVICE/APP، فایل main.py برای ایجاد برنامه و models.py، CORS برای تعریف جدول users، schemas.py برای DTOها، security.py برای crud.py، validate_token برای عملیات پایگاه داده و ROUTERS/Users.py برای endpointها قرار داده شده‌اند. مدل User شامل id هماهنگ با Auth، ایمیل یکتا، نام کامل، نقش، وضعیت فعال‌بودن و زمان ایجاد است.

4.3. مصرف JWT در Users

در این سرویس، توکن JWT به‌صورت محلی decode می‌شود و کاربر جاری از طریق get_current_user استخراج می‌گردد. این رویکرد مطابق الگوی Decentralized Token Consumption بوده و نیاز به تماس مکرر با Auth را کاهش می‌دهد.

4.4. Endpointها و همگام‌سازی داخلی

Endpointهای اصلی شامل /me (GET/PUT)، دریافت کاربر با شناسه، لیست کاربران and حذف کاربر است. همچنین یک مسیر داخلی api/users/internal/sync/ تعبیه شده که صرفاً با هدر X-Service-Key معتبر قابل دسترسی است و عملیات upsert کاربر را انجام می‌دهد.

5. جریان امنیتی انتها به انتها

- ۱) ثبت‌نام/ورود در سرویس Auth انجام شده و JWT صادر می‌گردد.
 - ۲) درخواست‌های بعدی همراه با Authorization: Bearer به API Gateway ارسال می‌شوند.
 - ۳) در گیت‌وی، توکن از طریق Auth verify می‌شود.
 - ۴) در صورت اعتبار، درخواست به سرویس مقصد فرووارد می‌گردد.
 - ۵) سرویس مقصد توکن را محلی decode کرده و کنترل دسترسی را اعمال می‌کند.
- بدین ترتیب، نیازمندی‌های امنیتی JWT و RBAC پروژه به‌صورت کامل محقق می‌شوند.

6. جمع‌بندی

در مجموع، درگاه API Gateway شامل قابلیت‌های درگاه واحد، CORS، میان‌افزار JWT، مسیریابی پراکسی، Circuit Breaker و Cache پیاده‌سازی شده است. در سرویس Auth نیز عملیات ثبت‌نام، ورود، نوسازی و صحت‌سنجی توکن‌ها، هش گذرواژه با bcrypt، مدیریت Refresh Tokenها و همگام‌سازی امن با Users انجام شده است. در سرویس Users، CRUD پروفایل، مصرف محلی JWT و همگام‌سازی داخلی امن به‌همراه upsert پیاده‌سازی گردیده است.